SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Dinesh R	**Reg. No.:**	192421117
Section / Batch:	B.Tech-IT	**Date:**	20/10/2026

Code of Conduct

I Dinesh R-192421117 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dinesh R

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

(a) Explain how Greedy Best-First Search behaves in this scenario. Discuss its strengths and weaknesses.

Greedy Best-First Search selects the path that appears closest to the destination using a heuristic function.

- **Strengths:** It is fast and useful when medicines must be delivered urgently.
- **Weaknesses:** It considers only the estimated distance and ignores the actual travel cost. It may select a blocked or dangerous route.

Therefore, it is fast but does not always find the safest or shortest route.

(b) Explain how A* handles the situation.

A* Search considers both the actual cost and the estimated distance to the destination.

Its evaluation function is:

f(n) = g(n) + h(n)

Where:

- **g(n):** Actual cost from the starting point to the current node.

- **h(n):** Estimated cost from the current node to the destination.
- **f(n):** Total estimated cost of the path.

A* can select a route that balances distance, travel time, and terrain conditions. It can avoid costly routes when accurate cost information is available.

(c) Analyze the impact of heuristic accuracy on both algorithms.

Heuristic accuracy affects the performance of both algorithms.

- In Greedy Best-First Search, an accurate heuristic helps the drone move towards the destination quickly.
- In A*, an accurate heuristic improves route selection and search efficiency.
- An inaccurate heuristic may direct the drone towards blocked or dangerous paths.
- An admissible heuristic helps A* find an optimal path when its other required conditions are satisfied.

Therefore, accurate heuristic information improves route planning.

(d) Discuss how dynamic changes affect search performance.

Floods and changing weather conditions may block previously available paths. The drone may need to recalculate its route when new information becomes available.

- Greedy Search may quickly select another route but can ignore its actual cost.
- A* can calculate a new route by considering updated travel costs.
- Repeated route calculations may increase processing time.
- Real-time updates help the drone avoid dangerous paths.

Therefore, the search algorithm should adapt to environmental changes.

(e) Recommend the most suitable algorithm and justify your decision.

*A Search is the most suitable algorithm** because it considers both actual travel cost and estimated distance. It can identify efficient routes while accounting for terrain and weather costs.

However, the drone must receive updated information about blocked paths and dangerous conditions. If a path becomes unavailable, it should recalculate its route.

A* provides a good balance between efficiency, route quality, and safety, provided that its cost model is accurate and the heuristic is appropriate.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

a) Formulate the problem as an optimization problem. Explain why traditional search is inefficient.

The objective is to optimize traffic signal timings to reduce waiting time, fuel consumption, and traffic congestion.

- **State:** Current traffic signal timings at all intersections.
- **Objective function:** Minimize total waiting time, fuel consumption, and congestion.
- **Possible actions:** Increase or decrease green-light durations and change signal timings.

Traditional search becomes inefficient because hundreds of intersections create a huge search space. Traffic conditions also change continuously, making it difficult to evaluate every possible solution.

(b) Explain how Hill Climbing works and identify its limitations.

Hill Climbing starts with an initial signal-timing arrangement and evaluates nearby solutions. It selects a better solution that reduces traffic congestion or waiting time.

Limitations:

1. It may get stuck in a local optimum.
2. It may fail to explore better solutions elsewhere.
3. It can struggle with changing traffic conditions.
4. It does not always find the global optimum.

Therefore, Hill Climbing is simple and fast but may not provide the best overall traffic solution.

(c) Explain local maxima, plateaus, and ridges with real-world examples.

1. **Local Optimum:** A signal arrangement reduces congestion at one intersection but increases congestion in nearby areas.
2. **Plateau:** Several signal arrangements produce almost the same traffic flow, making it difficult to identify a better solution.
3. **Ridge:** Improving traffic flow requires coordinated changes to multiple intersections, but changing only one signal does not improve the result.

These situations can prevent Hill Climbing from finding the best solution.

(d) Explain how Simulated Annealing or Genetic Algorithms overcome these limitations.

Simulated Annealing: It sometimes accepts a worse solution to escape a local optimum. As the search progresses, the probability of accepting worse solutions decreases.

Genetic Algorithm: It maintains a population of possible signal-timing solutions. Selection, crossover, and mutation generate improved solutions over multiple generations.

Both methods explore the search space more effectively than basic Hill Climbing.

(e) Recommend the best approach based on scalability and adaptability.

Genetic Algorithm is a suitable approach for optimizing traffic signals across hundreds of intersections.

It evaluates multiple solutions simultaneously and can explore different signal-timing combinations. Crossover and mutation help discover better arrangements and avoid becoming trapped in local optima.

The system can periodically update its population using new traffic data. This makes it useful for large and changing traffic networks.

However, computation time must be controlled to ensure that new signal timings are available when needed.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

(a) Explain why an online search agent is required instead of offline search.

An online search agent is suitable because the Mars rover does not have a complete map of the environment before exploration.

It observes its surroundings, takes actions, and learns about the terrain while moving.

Offline search requires sufficient information about the environment before planning. Since the rover encounters unknown obstacles, online search is more appropriate.

(b) Analyze the challenges of partial observability and dynamic environments.

The rover has limited sensing capability and cannot observe the entire environment at once.

The main challenges are:

1. Unknown craters, rocks, and other hazards.
2. Limited sensor accuracy.
3. Incomplete information about the terrain.
4. Uncertainty about the safety of unexplored paths.
5. The need to make decisions in real time.

These challenges make navigation difficult and require continuous observation.

(c) Describe how the rover builds and updates its internal model.

The rover uses sensors and cameras to observe its surroundings. It records discovered paths, obstacles, and safe areas in an internal map.

After moving to a new location, it collects more information and updates the map.

If it discovers a hazard, it marks that area as unsafe and searches for an alternative route.

This continuous learning helps the rover navigate safely through unknown terrain.

(d) Compare online search with uninformed and informed search strategies.

Feature	**Uninformed Search**	**Informed Search**	**Online Search**
Knowledge	No heuristic guidance	Uses a heuristic	Learns while exploring
Environment	Usually known in advance	Usually known in advance	May be unknown
Decision-making	Explores possible paths	Uses estimates to guide search	Acts and learns continuously
Adaptability	Limited in unknown environments	Depends on updated information	Adapts to new observations
Example	Breadth-First Search	A* Search	Rover exploring unknown terrain

Online search is more suitable when the rover must act without a complete map.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

An online search agent combined with heuristic guidance is the most suitable approach.

The rover can explore unknown areas, update its internal map, and avoid newly discovered hazards. Heuristic guidance can help it choose promising directions and reduce unnecessary exploration.

The rover should also prioritize safe paths and replan when new obstacles are detected.

This approach supports autonomous exploration under uncertainty while improving adaptability and safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

a) Formulate the problem as a CSP by defining variables, domains, and constraints.

A Constraint Satisfaction Problem consists of variables, domains, and constraints.

1. Variables:
Each exam is represented by a variable, such as Mathematics, Physics, or Computer Science.

2. Domains:
The domain contains the available examination time slots and suitable examination halls for each subject.

3. Constraints:

- Students must not have overlapping examinations.
- Examination halls must not exceed their capacity.
- Subjects must follow the required scheduling order.
- Faculty availability must be considered.
- Special accommodations must be provided where necessary.

The objective is to assign a suitable time slot and hall to every examination while satisfying all constraints.

(b) Explain how Backtracking Search works in this scenario.

Backtracking Search assigns time slots and halls to examinations one at a time.

If an assignment violates a constraint, the algorithm rejects it and tries another available option.

For example, if Mathematics and Physics have common students, they cannot be scheduled at the same time.

If no valid slot is available, the algorithm returns to an earlier assignment and changes it.

This process continues until a valid timetable is generated or all possible assignments are exhausted.

(c) Explain how constraint propagation, such as Forward Checking, improves efficiency.

Forward Checking reduces unnecessary searching by removing invalid choices from the domains of unassigned variables.

For example, after scheduling Mathematics at 9 AM, the system removes 9 AM from the available slots of other examinations that share students with Mathematics.

If any examination has no valid slots remaining, the system detects the conflict immediately and backtracks.

Therefore, Forward Checking reduces the search space and improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

The major challenges are a large number of examinations, limited halls, student conflicts, faculty availability, and special accommodations.

Backtracking Search with Forward Checking is a suitable strategy.

Backtracking finds assignments that satisfy the constraints, while Forward Checking detects conflicts early and reduces unnecessary exploration.

For larger universities, techniques such as the **Minimum Remaining Values (MRV) heuristic** can improve performance by selecting the examination with the fewest available time slots first.

This combination helps generate valid timetables more efficiently.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

a) Explain how the Minimax algorithm models this problem.

Minimax is an adversarial search algorithm used in games where two players have opposing objectives.

The AI is the maximizing player and tries to obtain the highest possible score. The human opponent is the minimizing player and tries to reduce the AI's score.

The algorithm explores possible moves and predicts the opponent's responses. It selects the move that provides the best outcome, assuming the opponent also plays optimally.

Therefore, Minimax helps the AI make strategic decisions.

(b) Analyze the computational challenges of large game trees.

Large game trees contain many possible moves and future game states.

The main challenges are:

1. High computational cost.
2. Increased memory usage.
3. Large branching factors.
4. Limited decision-making time.
5. Difficulty evaluating every possible game outcome.

Therefore, exploring the complete game tree may be impractical in real-time strategy games.

(c) Explain how Alpha-Beta Pruning improves efficiency.

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- **Alpha (α):** The best value found so far for the maximizing player.
- **Beta (β):** The best value found so far for the minimizing player.

When **α ≥ β**, the remaining branches of that part of the tree can be pruned because they cannot improve the decision.

For example, if the maximizing player already has a move worth 8 points and another branch cannot produce a result better than 5 points for that player, the unnecessary branch can be skipped.

Alpha-Beta Pruning produces the same optimal decision as Minimax when applied correctly, but it can examine fewer nodes.

(d) Design an evaluation function and justify the factors considered.

An evaluation function estimates how good a game state is when searching cannot continue to the end of the game.

A simple evaluation function is:

Evaluation Score = (3 × Resource Advantage) + (4 × Unit Strength) + (5 × Territory Control) − (6 × Damage to Base)

The factors are:

- **Resource Advantage:** Measures the difference in resources between the AI and opponent.
- **Unit Strength:** Measures the relative strength of available units.
- **Territory Control:** Rewards control over important areas.
- **Damage to Base:** Penalizes damage to the AI's base.

The weights represent the relative importance of each factor. They should be adjusted and tested according to the game's rules.

A higher score indicates a more favorable position for the AI.

(e) Recommend a strategy for balancing optimality and real-time performance.

Minimax with Alpha-Beta Pruning and a depth-limited evaluation function is the most suitable strategy.

Minimax evaluates possible moves, while Alpha-Beta Pruning reduces the number of states that must be examined.

Depth-limited search stops exploring after a fixed number of moves and uses the evaluation function to estimate the position.

Move ordering can improve pruning efficiency by examining promising moves first. An iterative deepening approach can also help the AI produce a valid decision within a time limit.